PRISM Team Brief — Fully Explained

A term-by-term, section-by-section walkthrough for someone with zero background in this material.

Before anything else: what IS this document?

This is a **technical project plan** for your hackathon team. It's not just "here's the problem" (you already have that) — it's "here's exactly what we're going to *build*, why every design choice was made, who does what, and in what order." It reads like it was written by someone (or an AI) with real EDA/VLSI/ML experience, so it's dense. That's normal — we'll unpack every piece.

The project is codenamed **PRISM** = **P**ower-integrity **R**isk **I**dentification, **S**lack-impact ranking and **M**itigation. That name is literally a summary of the 4-step pipeline you already know from before: **Find → Rank → Recommend → Quantify**, just using the fancier technical words: - Power-integrity Risk **Identification** = Find - **S**lack-impact ranking = Rank (using a specific, clever metric — explained in N2 below) - **M**itigation = Recommend + Quantify

---

SECTION 1: The decision in one paragraph — explained

This section justifies picking **PS1 over PS2**, which you already decided — but here's the reasoning it adds that's new and important:

> ***"Falsifiable"*** *means: a claim that can be definitively proven right or wrong with evidence. The document's core argument is: PS1's central claim ("we predicted the hotspots accurately") can be* ***proven*** *— you can run your prediction, then run a "real" solve, and directly compare the two, producing a table with actual error numbers. PS2's central claim ("we improved power/speed")* ***can't*** *be proven without literally building silicon and measuring it — you'd just be trusting a chain of assumptions nobody can check in a day. Under a tight deadline, you want to be judged on things you can prove, not things you can only argue.*

"Silicon" here just means an actual physical, manufactured chip — as opposed to a simulation or estimate.

The plan still gives you a PS2-flavored bonus at the end (§6.6, the "telemetry" closing module) so you get *both* stories without betting your whole project on the unprovable one.

SECTION 2: Why PS1 over PS2 — the comparison table, explained row by row

Row	What it means
Biggest rubric line	Reminds you of the heaviest-weighted grading criterion for each (25% each)
Can we prove that line?	PS1: yes, by comparing your prediction to a "real" solved answer. PS2: no, because proving a power/speed benefit needs actual measured hardware behavior, which you don't have
Ground truth	"**Ground truth**" = the correct, verified answer you compare your prediction against. For PS1, this is *a linear system we can*

	solve exactly — meaning the physics of voltage drop can be calculated with plain algebra/linear equations (no guessing needed, more below in §6.1). For PS2, you'd need to *invent* a thermal model (how heat spreads) and a process-variation model (how manufacturing differences behave) — both are estimates on top of estimates, less trustworthy
Judging question given?	PS1's judging slide gave you the exact quote to aim for (you saw this earlier: *"can it accurately identify… rank… recommend…"*). PS2 didn't give you that same exact target sentence, so you'd have to guess what "success" looks like
Day-1 output	How fast you get something visually impressive. PS1 gives you a heatmap almost immediately (visual, easy to explain "from across a room"). PS2 gives you a list of sensor locations, which needs more explaining to look impressive
Mitigation evidence	How solid your "here's proof the fix works" story is. PS1: you can literally **re-run the physics calculation** with the fix applied and show the number improve — that's *measured*. PS2: you'd only have a **narrative** (a story/argument), not a measured number
Crowdedness	How many other people are also solving a nearly identical problem. PS1 IR-drop prediction is a well-known research area — **PowerNet** and **IREDGe** are two specific published academic tools/papers that already do ML-based IR-drop prediction, and there was literally an **ICCAD'20 contest** (a real academic competition in 2020) about this exact problem. So many people have built something similar before, meaning your competitors (and judges) may already know standard approaches. PS2 is less explored, more "wide open," but for reasons already explained, wide-open = riskier under time pressure
Time to first result	Rough estimate of how many hours until you have *anything* working: ~4 hours for PS1 vs. ~12 for PS2

The honest counter-argument section is refreshingly transparent: it says *"if your team were unusually strong at optimization math rather than chip-design concepts, OR if you specifically knew everyone else was doing PS1 (making PS2 the 'blue ocean' choice), PS2 might actually be better."* Since neither is true for you, PS1 stays the right call. This is good, honest reasoning — not just cheerleading for one choice.

SECTION 3: What we are building — the 5 outputs, explained

1. **Predicted IR-drop hotspot map with an uncertainty band** — a heatmap of risky spots, but instead of saying "this spot will drop 37mV" (falsely precise), you say "this spot will drop somewhere between 31-48mV." That range is the **uncertainty band** — more honest and more useful, because it tells you how *confident* your prediction is.

2. **Risk ranking… ranked by picoseconds of timing slack actually lost, not by millivolts** — this is the single most important technical idea in the whole document (explained fully in N2 below). Short version: instead of ranking "which spot has the biggest voltage drop," you rank "which spot actually causes the chip to run late/fail," because those two things are **not the same** — a huge voltage drop somewhere that doesn't matter timing-wise is a non-issue, while a small voltage drop somewhere critical could break the chip.

3. **Mitigation recommendations… measured by re-solving the grid, ranked on a Pareto curve of benefit vs implementation cost** — instead of *guessing* how much a fix helps, you literally redo the physics calculation *with the fix applied* and see the actual improved number. **"Pareto curve"** = a chart showing all the "best possible" trade-off options between two competing goals (here: benefit vs. cost) —

more below in N3.

4. **A design-space exploration (DSE)** — instead of just patching problems in *one* floorplan design, you explore *many possible floorplan/design configurations* and find which one would have been the smartest to build in the first place. Explained fully in §8.
5. **TDET/PDET placement that closes the loop** — this is your PS2 "bonus" callback: use what you learned about IR-drop risk to smartly decide where PS2's sensors should go, tying the two problem statements together in one closing slide.

The one-sentence pitch, decoded

> *"IR-drop is not a voltage problem. It is a timing problem measured in volts — and once you convert millivolts into picoseconds, ranking becomes meaningful and mitigation becomes a budget optimisation instead of a guess."*

Translation: **Voltage drop by itself doesn't matter — what matters is whether it makes the chip too slow to meet its speed requirement.** Volts are just the "unit of measurement" of a problem whose *real* consequence is measured in time (picoseconds — trillionths of a second). Once you translate the problem into "how much time did we lose," you can rank problems by what actually matters, and turn "which fix should we apply" from a guess into a proper optimization/budgeting decision.

SECTION 4: Novelty — the five pillars, explained in full

This is the technical heart of the document — your actual "secret sauce" that differentiates you from other PS1 teams.

N1 — Physics-guided residual learning

The core idea: Don't build an AI model to predict IR-drop from scratch. Instead: 1. First, calculate an approximate answer using **known physics equations** (fast, exact, no training data needed). 2. Then, only train a small ML model to predict the **"residual"** — the *leftover error/gap* between that physics-based estimate and the true answer.

```
U_hat  =  U_coarse_solve  +  g(early features)
```

Translation: **Final predicted IR-drop = Rough physics-based estimate + (a small ML correction learned from data)**

Why this is smarter than a normal ML approach: A typical ML approach (like the published "PowerNet" tool) trains a neural network to predict IR-drop directly from a power map, with *no* physics built in. The network has to essentially "rediscover" basic electrical laws (Ohm's Law: Voltage = Current × Resistance) purely from limited example data — and if you show it a chip design shaped differently than what it trained on (different die size, different bump layout — **"bump"** = a physical connection point where power enters the chip from the package), it can fail badly, because it never actually *understood* the physics, just memorized patterns.

PRISM's approach instead says: *the exact physics equations already know Ohm's Law perfectly — no need to re-learn it.* What the physics-based rough estimate **can't** know at this early design stage is: - (a) exactly what power-wire pattern ("strap map") the detailed routing step will eventually build (decided later, not yet known) - (b) tiny local "hot spots" of current bunched together at a scale smaller than your rough estimate can see - (c) degradation of connections ("**vias**" — tiny vertical connectors between layers of wiring) near the edges of big components ("**macros**" — large pre-built functional blocks, like a memory module)

Since those three things are **local, small, statistical patterns**, that's the *only* thing you ask your ML model to learn — a much smaller, easier, more learnable task than "predict IR-drop from nothing."

“Measured today”: Their rough physics estimate alone gets an **$R^2 \approx 0.80$** (R^2 is a statistics measure of how well a prediction matches reality: 1.0 = perfect, 0 = useless — 0.80 is decently good already), but it’s *systematically* off by +5 to +11 millivolts (a reliable, consistent underestimate). That predictable gap is exactly what the small ML model needs to learn to close.

N2 — Volts → picoseconds (THE key differentiator)

Alpha-power law: `delay ∝ V / (V − Vth)^α` **- V** = the actual voltage a transistor receives **- Vth** (“threshold voltage”) = the minimum voltage needed before a transistor can switch on at all **- α (alpha)**, roughly 1.3 here, is an empirical exponent (a number found from measurement, describing how sensitive the transistor is to voltage changes) - This formula says: **delay** (how long a transistor takes to switch) gets **worse** (bigger) as voltage V drops closer to the threshold Vth.

The sensitivity number: `d(ln delay)/d(ln V) ≈ −0.95` — don’t worry about the calculus notation; the plain-words takeaway is: **roughly, a 1% drop in voltage costs you about a 1% increase in delay.**

The actual algorithm: For every “path” in the chip (a **path** = a chain of connected logic elements a signal travels through, from a starting point to an ending point, within one clock cycle), you: 1. Look at how much voltage droop hits each cell along that path 2. Convert each cell’s droop into “how much extra delay did that cost,” using the formula above 3. Add up all those small delays along the whole path 4. Subtract that total from the path’s **“slack”** — **slack** = how much spare time margin a path has before it becomes too slow (if slack goes negative, the chip fails to meet its speed target on that path) 5. What’s left is called **“effective slack”** — the *true*, voltage-drop-adjusted timing margin

Why this matters so much: > *“40 mV of droop where there is 500 ps of slack is a non-issue. 15 mV on a critical path is a respin.”*

Translation: A **big** voltage drop (40mV) somewhere with tons of spare timing margin (500 picoseconds) doesn’t actually matter. A **small** voltage drop (15mV) on a path with almost zero spare margin (a “**critical path**” — closest to failing) could be enough to break the chip and force an expensive **“respin”** (redesigning and re-manufacturing — extremely costly).

If you only ranked by millivolts (the “obvious” way most teams will do it), you cannot tell these two situations apart. Ranking by effective slack (picoseconds actually lost) *can*. This is explicitly your single strongest differentiator, and it directly answers the “risk assessment and prioritisation” scoring criterion (20% of your grade).

N3 — Mitigations measured, not asserted

“Asserted” means “claimed without proof.” Most teams will just say something like “adding a decoupling capacitor typically helps by about 10%” — a rule of thumb, not actually calculated for *your specific* chip.

PRISM instead **literally re-runs the physics calculation** with each proposed fix mathematically applied, and reads off the *actual* new (improved) number. This works because the voltage drop math is **exactly linear** in current, meaning you can precisely calculate the effect of removing/reducing current without needing to re-simulate the entire chip from scratch (cheap and exact, not an approximation).

Knapsack problem: a classic optimization concept — imagine a backpack with limited capacity (your budget: area, routing space, engineering effort) and a pile of items (your possible fixes), each with its own “weight” (cost) and “value” (benefit). The **knapsack algorithm** figures out the best combination of items without exceeding capacity — “given our limited budget, which combination of fixes gives us the most total benefit?”

Sweeping the budget → Pareto curve: solving the knapsack problem repeatedly for many different budget sizes and plotting the *best possible outcome at each budget level* produces a **Pareto curve/front** — a line showing the best trade-off achievable at every point between “cheap and modest benefit” and “expensive and maximum benefit.” This directly matches the official judging question’s phrase: *“a measurable benefit-to-cost advantage.”*

N4 — The surrogate is what makes DSE possible

“Surrogate” = a fast stand-in/approximation for a slow, expensive process. Here, your ML prediction model stands in for a *real* full chip-design tool run.

- A **real** full physical-design + signoff analysis of one design configuration takes about **20 minutes**.
- PRISM’s fast predictor takes about **50 milliseconds**.
- That’s roughly a **24,000× speedup**.

This makes something otherwise impossible in a hackathon timeframe actually feasible: **exploring thousands or even hundreds of thousands of different possible chip design configurations** to find good ones — the **Design-Space Exploration (DSE)** from §3’s 4th deliverable, fully explained in §8.

N5 — Scenario economics

Instead of treating every scenario as equally important, **weight each scenario by how much of the chip’s real-world runtime it actually represents**, then report the “expected” (probability-weighted average) risk.

Example: *“GC compaction is 10% of runtime but 60% of expected IR risk.”* (**“GC compaction”** = garbage collection compaction, a specific heavy-workload operation common in storage/SSD controllers). Even though this operational mode only happens 10% of the time, it’s responsible for the majority (60%) of your chip’s *expected* voltage-drop risk — meaning it deserves outsized engineering attention despite being rare.

This is a stronger, more nuanced answer to the “rank operating scenarios” requirement than what most teams will likely do: rank scenarios by simple peak power (the biggest instantaneous number) rather than this smarter “risk × how often it actually happens” weighting.

Bonus: the scalability argument

“Conductance matrix” — the mathematical representation of the chip’s power-wiring network as a system of equations (more in §6.1). **“Factorises once”** — a mathematical/computational trick: you can do the expensive part of the math **one single time**, and then each new scenario (idle/peak/etc.) only requires a cheap, fast additional step, rather than redoing the expensive part every single time.

Commercial EDA tools typically **re-run the entire expensive analysis separately for every single scenario (“corner”)** — much slower. PRISM’s approach is claimed to be **10-50× faster** at this specific part.

“Algebraic multigrid” and **“Laplacians”** are advanced math/computer-science terms about efficiently solving large systems of equations — you don’t need to deeply understand these to use the argument; the takeaway is “our math structure scales very efficiently to large, complex chips,” worth a slide mention but not something you need to personally master.

SECTION 5: Architecture diagram — explained block by block

Reading top to bottom, this is the **pipeline** — the order data flows through your system:

1. **Parameterised RTL** — your starting chip design, written in a hardware description language (SystemVerilog), with adjustable “knobs” (explained fully in §7)
2. **Yosys** — a free, open-source tool that converts your RTL design into a “**netlist**” (the list of logic components and how they’re wired together — not yet given physical positions)
3. **OpenROAD / ORFS** — a free, open-source chip physical-design toolchain that takes the netlist and does **floorplanning** (deciding where blocks physically sit), **PDN** (Power Delivery Network — designing the actual power wiring), **placement** (positioning individual components), **CTS** (Clock Tree Synthesis — designing the wiring that distributes the clock signal), and **routing** (drawing the actual wires)
4. This produces two separate streams:
    - **“Early artefacts”** (DEF file = a standard format describing physical layout, plus reports and the netlist) — your **input data**, representing what you’d realistically have at an early design stage
    - **“PDNSim IR map”** — your **ground truth/label** — the actual, correct IR-drop answer, calculated by a proper tool, used *only* to check your predictions against, never as an input to your model

5. **B1 Features** — extracting ~30 measurable characteristics from the early-stage data (§6.3)
6. **B1' Ground Truth** — a separate, more detailed/accurate physics calculation used to generate the "correct answer" for training/testing
7. **B2 Hybrid Predictor** — combines the rough physics estimate with a small ML correction (N1)
8. **B3 Risk Engine** — converts predicted voltage drop into "effective slack" (picoseconds lost) and ranks accordingly (N2)
9. **B4 Mitigation + ROI** — generates and scores fix recommendations (N3). **"ROI"** = Return On Investment, benefit relative to cost
10. Branches into three final outputs:
    - **B5 DSE** — explores many design configurations (N4, detailed in §8)
    - **B6 Telemetry** — the PS2-style sensor placement bonus (§6.6)
    - **B7 Dashboard** — your visual demo interface

The critical honesty rule: *"anything derived from the as-built grid or the signoff solve is a label, never a feature."* You must never let your prediction model "see" the correct answer while making its prediction — that would be cheating (called **"data leakage"** in ML). There's even an **automated checker** (`prism/audit.py`) built specifically to catch this mistake, and it's suggested you run this check live in front of judges if asked, to prove your model isn't cheating.

SECTION 6: The solution, block by block — explained in depth

6.1 — B0: Physics engine

The core mental model: Simplify the chip's power-wiring network into a grid — imagine a checkerboard where each square ("**tile**") is one node, connected to its neighbors, with each connection having a certain "**conductance**" (how easily electricity flows through it — the opposite of resistance), plus a connection from certain tiles down to the power supply itself (via "**bumps**").

KCL = Kirchhoff's Current Law, a basic, universally true electrical engineering rule: at any junction, current flowing in must equal current flowing out. Applying this rule across your whole simplified grid produces one big system of linear equations:

```
(L + D_bump) · U = I
```

You don't need the exact matrix math. In plain English: **this equation lets you calculate exactly how much voltage drop (U) occurs at every tile, given how much current (I) each tile is drawing** — and because it's a "linear system," it can be solved directly and exactly with standard math tools (no guessing, no approximation error).

Two properties this unlocks: - **"U is exactly linear in I"**: if you change how much current a tile draws (e.g., a mitigation fix reduces current there), you can calculate the *exact* new voltage drop instantly, without re-solving the whole system or introducing approximation error. This is *why* the "measured, not asserted" mitigation approach (N3) works cheaply and reliably. - **"Scaling every conductance by s scales U by 1/s"**: if you uniformly strengthen your entire power grid (thicker wires everywhere), you can calculate the new result with simple division, again without re-solving everything ("**closed-form**" means a direct formula, no trial-and-error search needed).

The clever framing: this simplified grid mesh itself counts as the "additional artefact/metric you derived" that both problem statements explicitly invite you to justify (the "discovery expectation" bullet from the Inputs slides). Say this out loud in your presentation — it shows you responded to a subtle part of the brief other teams might miss.

6.2 — B1: Two fidelities, and the gap between them

"Fidelity" = how detailed/accurate a version of the data is. PRISM deliberately creates **two different-quality**

versions of the same design:

	Ground truth (the “correct answer”)	Early estimate (what you’d realistically predict from)
Grid detail	Fine (detailed)	Coarse (rough)
Strap map	As-actually-built	Preliminary/rough guess
Current data	Per individual component	Averaged per tile

The **gap** between these versions is deliberately caused by the *same real-world reasons* that make early IR-drop prediction genuinely hard: routing congestion later forces power wires to be thinner than planned, connections near big components degrade unpredictably, and current bunches up in areas smaller than your rough estimate can even see.

Key insight: *“If there were no gap, early IR prediction would already be a solved problem.”* The entire reason this is a hard, worthwhile problem is *because* early-stage data is necessarily incomplete/rough compared to the final truth. Your job is to bridge that gap intelligently.

6.3 — B2: Hybrid predictor

Gradient-boosted trees: a well-established ML technique (an alternative to neural networks) that builds a prediction by combining many small “decision trees” (simple if/else rules) together, each correcting the mistakes of previous ones. Easier to get working well with limited data than a full neural network — a sensible, safe choice for a time-limited hackathon.

`HistGradientBoostingRegressor`: the specific tool from the `scikit-learn` Python library, chosen because it doesn’t require installing extra software packages (`xgboost` is a popular but separately-installed alternative) — reducing installation-failure risk during your limited time.

~30 early-only features, in seven groups (`phys_*`, `grid_*`, `cur_*`, `conc_*`, `top_*`, `sw_*`, `scn_*`): your actual input measurements, organized into categories — physical characteristics, grid/wiring characteristics, current-related characteristics, concentration (how clustered activity is), topology (physical layout/shape), switching activity, and scenario information. Person B will define these precisely.

10th and 90th percentile quantile models: instead of one single predicted number, produce a **range** where you’re roughly 80% confident the true answer falls between the low (10th percentile) and high (90th percentile) estimates. This is what produces the “31-48mV, budget is 45” style output — the *width* of that range tells you how confident your model is about that spot.

Validation rules — non-negotiable: - **“Design-level holdout, never a random tile split”**: test your model on an *entirely separate design* it’s never seen, not just random individual tiles from designs it *has* seen. Neighboring tiles on the same chip tend to be very similar/correlated — so randomly mixing tiles from the same design into both training and testing sets gives an artificially inflated, misleading accuracy score. Testing on a whole separate, unseen design is the honest way to check if your model actually generalizes. - **“Headline metric = top-5% hit rate, not MAE”**: **MAE (Mean Absolute Error)** is a common but abstract statistic (average error size). PRISM argues the metric that actually matters practically is: *“out of the actual worst 5% riskiest tiles, how many did your model correctly flag?”* — because that’s what a real physical-design engineer would act on: a shortlist of tiles to inspect, not an abstract average error number. - **“Always report the ablation”**: an **ablation study** tests multiple versions of your system — physics-only (no ML), ML-only (no physics), and the combined hybrid — to *prove* combining them is actually better than either alone, rather than just asserting it.

6.4 — B3: Risk engine

The practical implementation of N2 (volts → picoseconds), as concrete steps: 1. Convert predicted voltage droop into added delay, using the alpha-power law formula 2. Sum those delays along each full timing path to get “effective slack” (true, adjusted timing margin) 3. Figure out which specific tiles/locations are responsible for how much of that lost slack (“**attribute**” the blame back to the source) 4. **Roll up** (aggregate/summarize) results

from individual tiles up to bigger groupings: tile → block → power domain — this matters because the problem statement specifically asks you to rank blocks and power domains, not just individual tiny tiles 5. Apply the scenario-weighting from N5 to get final "expected risk" numbers

Stated goal: *"a ranked table a physical-design lead could act on Monday morning"* — genuinely practical and actionable, not just an academic exercise.

6.5 — B4: Mitigation catalog and ROI

Concrete fix options, each with its physical mechanism and cost type:

Fix	What it physically does	What it costs
Decap insertion	Adding small "decoupling capacitors" — tiny components that act like local mini backup-batteries, smoothing out sudden current spikes	Extra chip area, extra background power leakage
Strap widening	Making the power wires ("straps") physically wider/thicker, so they carry current with less resistance	Uses up routing space
Cell de-densification	Spreading components out more so current demand isn't as concentrated	Chip area, wire length, timing
Extra bump/pad	Adding another physical connection point where power enters the chip	Expensive and limited — physical package connector pins are scarce
Vt swap / downsize (non-critical only)	Swapping to components with a different threshold-voltage characteristic or making them smaller, specifically only on paths that aren't timing-critical, to reduce current draw	Costs some timing safety margin
Clock-skew staggering	Deliberately making different chip parts switch at slightly offset times (instead of all at once) so they don't all draw peak current simultaneously — smooths the total current demand curve	Clock-tree design effort, extra small buffer components

"Clock-skew staggering is the cheap clever one nobody else will propose. Lead with it in the presentation." — flagged as your standout, memorable idea, because it requires understanding *why* current spikes happen (many things switching simultaneously) rather than just the obvious "add more wire/capacitor" fixes.

Each fix candidate gets scored by literally re-solving the physics grid with it applied (6.1), then the knapsack algorithm (N3) picks the best combination under a given budget, and sweeping across many budget sizes produces your Pareto curve.

6.6 — B6: Telemetry extension (the PS2 bonus)

Turns your PS1 IR-drop risk findings into a **"prior"** (an informed starting guess) for where PS2's TDET/PDET sensors should be placed — connecting both problem statements: put your real-time monitoring sensors specifically where your analysis already told you the risk is highest.

"Submodular maximum-coverage" and **"greedy has a provable 1−1/e bound"**: imagine choosing a limited number of sensor locations, where each sensor "covers" (usefully monitors) some region around it. A **"greedy" algorithm** (repeatedly pick whichever next sensor location adds the most new coverage) has a mathematically proven guarantee — provably at least about 63% ($1 - 1/e \approx 0.63$) as good as the theoretically perfect placement, while being much simpler and faster to actually run. A nice, defensible technical detail if a judge asks "how do you know your sensor placement is any good?"

Explicitly flagged as low-effort-high-value: "**one slide, ~90 minutes of work**" — a bonus closing flourish, not

your main deliverable.

SECTION 7: Parameterised RTL design — explained

Why this section exists: Your biggest weakness, honestly acknowledged, is not having much real chip design data to test on. The solution: build **one design with adjustable settings ("parameters")**, so you can generate *many different variations* of essentially the same chip, each producing its own genuinely real netlist, floorplan, timing report, and IR-drop map. This turns "we only tested on 3 examples" (weak) into "we swept across a whole design space" (much stronger claim) — and it's the raw material for the Design-Space Exploration in §8.

RTL = Register Transfer Level — the actual *logic design* of your chip, written in a hardware description language (here, **SystemVerilog**) — the "blueprint code" describing what the chip does, before it becomes physical geometry.

The example design: `ssd_ctrl_top`

A made-up but realistic **storage controller** chip (manages an SSD/flash storage device) — chosen because it's complex enough to have genuinely interesting, uneven switching activity (realistic IR-drop patterns) but structured/regular enough to build quickly.

Its sub-components: - `host_if` — interface talking to the outside host computer, using a standard protocol called **AXI4-Lite** - `cdc_fifo` — "Clock Domain Crossing FIFO" — a small buffer safely passing data between two parts of the chip running on different clock timings (a genuinely tricky, important design challenge) - `dma_fabric` — Direct Memory Access fabric — traffic-routing logic ("**crossbar**" = a switch connecting any input to any output, "**arbiter**" = logic deciding who gets priority when multiple things want access at once) - `ecc_engine` — Error Correction Code engine — checks/fixes data errors, marked "← hot" meaning expected high-activity/high-power (a likely IR-drop hotspot candidate) since it constantly does intense math (**GF(2) syndrome/XOR tree** — specific error-correction math, details not important) - `ch_ctrl[NUM_CH]` — one controller per NAND flash memory channel (physical connections to flash storage chips) - `sram_ctl` — controls an on-chip memory buffer (fast memory type) - `seq_core` — a small internal sequencer/processor-like component

The parameter file, explained

```
        parameter int NUM_CH      = 4;    // NAND channels        {2, 4, 8}
```

This makes a design "adjustable" — instead of hard-coding exactly 4 memory channels, define it as a **parameter** that can be set to different values (2, 4, or 8) when generating each variation. Same idea applies to data width, error-correction complexity, buffer depth, pipeline depth, memory bank count, and whether fine-grained clock gating (a power-saving technique that shuts off clock signals to idle circuit parts) is enabled.

Rules for whoever writes this RTL — all about discipline needed to make the "sweep many configurations" idea work: - Never hard-code a fixed size anywhere (e.g., never write `[31:0]` directly) — always reference the parameter, or the whole sweep breaks - Use `generate ... for` loops — a SystemVerilog feature letting you automatically create multiple repeated copies of hardware (e.g., one FIFO buffer per memory channel) based on a parameter's value - Must be **"Yosys-synthesisable"** — written using only the subset of SystemVerilog features the free `Yosys` tool can actually process - **"Lint with Verilator"** — **linting** means running an automated checker (`Verilator`) that catches syntax/logic mistakes before you waste time trying to synthesize broken code - Clock-gating enable paths matter because they create realistically *different* activity patterns between scenarios (idle vs. busy) — without this, your "scenarios" wouldn't actually look meaningfully different - Keep a testbench (`tb_ssd_ctrl.sv`) driving your six scenarios, producing a **VCD** file (Value Change Dump — a standard format recording exactly when every signal changes over simulated time) — giving you **real, measured switching activity data** instead of guessing/estimating it

Generating a configuration — the commands, explained

```
python3 scripts/gen_config.py --config configs/cfg_0042.json --out rtl/params.svh
yosys -p "read_verilog -sv rtl/*.sv; synth -top ssd_ctrl_top; write_verilog out/netlist.v"
```

Line 1: a script reads a settings file (JSON, a simple structured text format) and writes the actual parameter file for that specific configuration. Line 2: runs `Yosys`, telling it to read your Verilog source files, synthesize (convert design logic into actual netlist components) the top-level module `ssd_ctrl_top`, and save the resulting netlist to a file.

Physical knobs table

A second layer of adjustable settings — not the *logic* design (RTL), but *physical* implementation choices: how densely packed the chip is (`CORE_UTIL`), its physical proportions (`ASPECT_RATIO`), spacing/thickness of power wires (`PDN_STRAP_PITCH_UM`, `PDN_STRAP_WIDTH_UM` — "UM" = micrometers), which wiring layers carry power, spacing of physical power bumps, how much decoupling-capacitor filler is inserted, and clock speed target. Combined with RTL parameters, this gives a large space of genuinely different possible chip configurations — feeding directly into §8's DSE.

SECTION 8: Design Space Exploration (DSE) — explained

What problem this actually solves

Earlier blocks (B2-B4) answer: *"given this one specific floorplan, where's the problem and what's the cheapest patch?"* DSE asks the bigger question: **"out of all the possible ways we could have configured this chip, which configuration should we have built in the first place?"** This reframes your project from "a tool that finds and patches problems" into "a tool that helps make better design decisions upfront" — a more ambitious, impressive positioning.

Why this is only possible because of your fast surrogate model

Reiterating N4's numbers: a real full physical-design run takes ~20 minutes, letting you test only about 24 configurations in 8 hours. Your fast (50-millisecond) surrogate model lets you test roughly **500,000** configurations in that same time — the entire justification for why building a learned prediction model is worth doing here.

The three-tier strategy

Tier 1 — Screening (Sobol sampling): **Sobol sampling** is a specific, mathematically well-distributed way of randomly sampling many combinations across your 13 adjustable "knobs," designed to spread test points evenly rather than clustering randomly. You run 512-2048 sampled configurations through your fast model, then compute **"sensitivity"** — which knobs actually matter most, and which barely matter. The document predicts, before you run it: strap pitch (power wire spacing) and bump pitch (power connection spacing) will probably matter most, while ECC lane count will matter only indirectly through its effect on local current concentration — a confident, testable prediction worth including in your presentation.

Tier 2 — Multi-objective optimisation (NSGA-II): **NSGA-II** is a well-established genetic-algorithm-style optimization technique (loosely inspired by biological evolution — testing many candidates, keeping the best, combining/mutating them, repeating) designed for problems optimizing **multiple competing goals at once** (here: minimize peak voltage drop, minimize timing slack lost, minimize chip area, minimize routing resource usage, minimize total power — five conflicting goals simultaneously). `pymoo` is the specific free Python library used to run this algorithm.

The output is again a **Pareto front** (as in N3) — but showing best trade-off combinations across *entire chip configurations*. **"Hypervolume"** is a mathematical way to measure "how good is this whole set of trade-off points, as a single number" — useful for proving your optimization found meaningfully better results than blindly guessing or exhaustively trying every combination ("full-factorial search" — usually too slow to be practical).

Tier 3 — Bayesian optimisation for the expensive runs: Since you can only afford maybe 12-16 *real* (slow, ~20-minute) physical-design runs total, you don't want to pick those randomly. **Bayesian optimization** with a **GP (Gaussian Process — a statistical model good at estimating uncertainty)** and **"Expected Improvement"** (a strategy balancing "try something we're confident will be good" against "try something uncertain that might be even better") is the established technique for making this smart, limited-budget choice. You then feed the results of those real runs back into retraining your fast surrogate model, making it progressively more accurate — a **"closed loop"**: cheap model suggests → expensive real tool verifies → model learns and improves. This mirrors how real professional chip-design methodology teams work.

SECTION 9: Software to install — explained

- **WSL (Windows Subsystem for Linux)**: if on Windows, this runs a real Linux environment (Ubuntu) inside Windows, because most chip-design tools are built primarily for Linux.
- **Docker Desktop**: runs software in isolated, pre-packaged "containers" — useful since complex EDA tools often have complicated installation requirements Docker can sidestep.
- **Python + libraries**: `numpy` / `scipy` (math/numerical computing), `pandas` (data tables), `scikit-learn` (ML, including B2's gradient-boosting model), `matplotlib` / `plotly` (charts), `streamlit` (your dashboard), `pymoo` (the NSGA-II library from §8), `networkx` (graph/network math), `pyyaml` (reading configuration files), `joblib` (saving/loading trained models). `xgboost` / `lightgbm` are explicitly avoided to reduce installation risk.
- **Yosys, Verilator, Icarus, GTKWave**: the free chip-design toolchain (synthesis, linting, simulation, waveform viewing). **GTKWave** specifically visually inspects the VCD waveform files from §7.
- **OpenROAD-flow-scripts (ORFS)**: the free physical-design toolchain (floorplan → PDN → place → route). Flagged as "**the important one**" — the most critical, likely most complicated, piece to get working.
- **The `gcd` smoke test**: "gcd" (Greatest Common Divisor) is a small, standard example design bundled with OpenROAD specifically to test your whole toolchain installation actually works end-to-end before you trust it with your real project — a "**smoke test**" catches major breakage early.
- **What you actually need out of ORFS**: the routed **DEF** file (physical layout description), the **OpenSTA** timing reports (STA = Static Timing Analysis — checking if all signal paths meet speed requirements without simulating), the synthesized netlist, and critically — the **PDNSim** voltage map, which becomes your actual **ground truth** to train and validate your prediction model against.
- **PDKs (Process Design Kits)** — Nangate45 and Sky130 are free, publicly available "process kits" (standardized descriptions of a manufacturing process's physical/electrical characteristics) letting you run realistic chip-design flows without proprietary manufacturer data. Bundled with ORFS.
- **Cadence Virtuoso** — explicitly **optional, not on the critical path** — professional, expensive software mainly for detailed analog/custom circuit design. Only one small optional simulation suggested if you have spare time (showing a signal slowing under voltage droop via a professional simulator called **Spectre**, visually connecting to the alpha-power law from N2) — explicitly "worth zero if we're behind" — skip it if short on time.

SECTION 10: What to learn, by role — explained

A **study checklist**, mapped to your 4 team roles (§11). Notable terms not fully covered above:

- **STA (Static Timing Analysis)**: checking whether every signal path meets timing requirements. **WNS** = Worst Negative Slack (the single worst/most-failing timing path's margin), **TNS** = Total Negative Slack (sum of *all* failing paths' margins) — standard chip-timing-analysis metrics. **Setup/hold** are two specific types of timing requirements a signal must satisfy (arriving neither too late nor too early relative to the clock).
- **Graph Laplacian / KCL as a linear system**: the deeper math behind §6.1's physics engine — `L·1 = 0` is a mathematical property (a specific matrix, multiplied by an all-1's vector, gives zero) reflecting a physical truth: if every point in the circuit were at the exact same voltage, there'd be no current flow anywhere — this is what "removes the supply term" in solving the equations. Person B needs this to correctly build the physics solver.

- **Overfitting/underfitting, holdout design**: standard ML concepts — **overfitting** means your model memorizes training examples too specifically and fails on new data; **underfitting** means it's too simple to capture real patterns; a good **holdout** (separate test) design, per 6.3, is what lets you honestly check which problem you have.
- **Submodularity**: the mathematical property underlying the greedy sensor-placement algorithm from §6.6 — a fancy word for "diminishing returns" (each additional sensor adds less new coverage than the previous one did), which is what allows the 1-1/e guarantee to hold.

SECTION 11: Work distribution — explained

The four roles, in plain terms

- **A — RTL & Flow owner**: builds the actual adjustable chip design (§7) and runs it through the whole toolchain (Yosys → ORFS) to produce real design data
- **B — Physics & Prediction owner**: builds the core physics solver (§6.1) and the hybrid ML prediction model (§6.2-6.3) — arguably the most technically demanding role
- **C — Risk, Mitigation & DSE owner**: builds the volts→picoseconds conversion (N2), the mitigation catalog and cost optimization (N3), and the design-space exploration (§8)
- **D — Product owner**: builds the visual dashboard, prepares slides/report, writes and rehearses the demo script, and is specifically responsible for periodically running the *entire* pipeline end-to-end to catch integration problems early

The interface files — why this matters so much

Instead of everyone's code directly calling/depending on everyone else's half-finished code (causing constant breakage and merge conflicts), **everyone agrees upfront on a set of simple file formats** (plain CSV/npz data files) written to disk at each pipeline stage. Person A doesn't need to understand Person B's code — they just produce a file matching the agreed format, and Person B reads it. This lets all four work genuinely in parallel without blocking each other — a practical, battle-tested approach to fast collaborative work under time pressure.

"Everything is precomputed to disk. The dashboard never trains or solves live." — an important, deliberate safety decision: your final demo should only ever *display* already-calculated results, never try to run a live calculation during the actual presentation. This avoids your demo crashing or hanging in front of judges.

The timeline table

Lays out roughly a 35-hour window (map this against your *actual* available hours, since your "till 11pm tomorrow" constraint might be shorter). It explicitly **schedules a sleep block** (T+22 to T+28) — a genuinely important piece of practical wisdom: exhausted teams make more mistakes and work slower, so planned rest is a real part of the strategy.

If you're fewer than four people

With 3 people, merge "Product" into whoever's furthest ahead, since presentation/dashboard work is more compressible than the core physics work. With 2 people, split into (Data+Prediction) and (Risk+Mitigation+Product), cutting the most advanced/optional pieces (DSE tier 3, telemetry module) but **keeping** the volts→picoseconds conversion regardless — it's both your biggest differentiator (N2) and small to implement (~150 lines).

Priorities if you fall behind — the cut list

Your **emergency triage plan**, ordered from most-essential (top, P0) to least-essential (bottom, P2), with the instruction "cut from the bottom first." Drop the telemetry module and advanced DSE tiers *before* ever cutting the core physics engine, hybrid predictor, risk ranking, mitigation catalog, or slides/rehearsal — those five P0 items are non-negotiable, because they're what the grading rubric actually measures.

SECTION 12: Definition of done — the demo script, explained

Your actual presentation, in under 4 minutes:

1. **The setup**: establish a baseline — show a scenario (read-stream) where your chip currently passes safely (42mV droop against a 45mV budget)
2. **The reveal**: switch to a different scenario (GC compaction) and show numbers get dramatically worse (65mV — over budget). Flagged as "**the moment that lands**" — your key attention-grabbing beat, showing scenario matters and a static, single-scenario analysis would have missed this danger entirely
3. **The prediction**: show your predicted map next to the "real" signoff map side-by-side, with your top-5% hit-rate accuracy number displayed
4. **The ranking**: show the millivolt-based ranking next to your effective-slack-based ranking, pointing out a case where they *disagree* — a high-voltage-drop spot that turns out not to matter, and a low-voltage-drop spot that turns out to matter a lot. Your core differentiator (N2), made visually obvious and memorable
5. **The fix**: pick your top-ranked risky spot, show several possible fixes with actual measured benefit, and show the knapsack algorithm automatically selecting the best combination within a budget
6. **The curve**: show your Pareto front chart, with a punchy soundbite ready: *"40% of the droop for 12% of the cost"*
7. **The bigger answer**: show your Design-Space-Exploration results — maybe the better solution isn't patching this floorplan at all, but choosing a fundamentally different, better configuration from the start
8. **The close**: show your telemetry/sensor-placement map, tying back to the hackathon's own overall title/theme about turning distributed silicon sensor data into intelligent optimization

---

SECTION 13: Anticipated judge questions — explained

A pre-written FAQ/defense, showing you've thought critically about your own project's weaknesses:

- **"Isn't this just PowerNet / an ML predictor?"** → No — your prediction step is only the *input* to your actual contribution (risk ranking, mitigation optimization, design-space exploration built on top of it), not the whole project.
- **"Isn't your ground truth circular (your own solver checking itself)?"** → Honestly: yes, partially, for your synthetic/generated data — exactly why you also incorporate *real* OpenROAD/PDNSim results, and why your "residual learning" approach (N1) specifically limits the model to learning only small corrections on top of real physics.
- **"Would this work on a real modern chip (5 nanometer)?"** → Your general *approach/method* would transfer, but specific trained model weights would need retraining using a real company's actual historical chip data — an honest, appropriately humble answer avoiding overclaiming to a judge who might be a real industry expert.
- **"Why trust a 50ms prediction over a 20-minute 'real' tool run?"** → You shouldn't blindly trust it instead — it's a **screening tool**, telling you *where* to spend limited real, expensive analysis time, not replacing that analysis entirely. The uncertainty band (B2) supports this honest framing.
- **"What's your biggest weakness?"** → Your analysis only covers **static** IR-drop (steady-state, average-conditions view), while **real** voltage droop is also affected by fast-changing dynamic effects (rate of current change interacting with the physical inductance of the chip's packaging, with capacitors acting as a limited local reserve of charge) — properly modeling that would require more advanced "transient" (time-varying) analysis, out of scope here. Naming your own biggest limitation *before* a judge points it out is a deliberate, credibility-building strategy.

Final summary: what you actually need to internalize before building

1. **Voltage drop only matters if it costs you timing margin** — rank by picoseconds lost, not millivolts

(your single biggest differentiator).

2. **Predict only what physics can't already tell you exactly** — combine a fast, exact physics calculation with a small ML model that only learns the leftover gap, rather than learning everything from scratch.
3. **Never assert a fix's benefit — calculate it**, by literally re-solving your physics model with the fix applied.
4. **Everything for the demo should be pre-calculated and saved to disk** — never make your live presentation depend on something running/calculating in real time.
5. **Know your own weaknesses and say them out loud before a judge does** — it reads as more credible, not less.